MLA02 – FUNDAMENTALS OF MACHINE LEARNING
QUESTION 3

Decision Tree Classification Using the ID3 Algorithm

Introduction

ID3 builds a decision tree by selecting attributes with high information gain. This lab uses an entropy-based decision tree to classify new categorical samples.

Implementation library / technique: DecisionTreeClassifier(criterion='entropy').

General workflow: prepare the data, separate features and target labels, train the model, classify a new sample, and display the result.

Problem 1: Play Tennis Prediction

Aim

To implement Decision Tree Classification Using the ID3 Algorithm for play tennis prediction and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Outlook, Temperature, Humidity, Wind and their class labels to train a model, then classify a new sample.

Algorithm

1. Import the required Python libraries.
2. Store the training samples in a feature matrix X and labels in y.
3. Create the classifier and fit it using X and y.
4. Prepare the new sample and apply the same preprocessing, if required.
5. Predict and display the class of the new sample.

Input (Training Data)

Outlook	Temperature	Humidity	Wind	Target
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No

Program

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
import numpy as np

attributes = ['Outlook', 'Temperature', 'Humidity', 'Wind']
training_data = [['Sunny', 'Hot', 'High', 'Weak', 'No'], ['Sunny', 'Hot', 'High', 'Strong', 'No'], ['Overcast',
'Hot', 'High', 'Weak', 'Yes'], ['Rain', 'Mild', 'High', 'Weak', 'Yes'], ['Rain', 'Cool', 'Normal', 'Weak',
'Yes'], ['Rain', 'Cool', 'Normal', 'Strong', 'No'], ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'], ['Sunny',
'Mild', 'High', 'Weak', 'No']]
X_raw = [row[:-1] for row in training_data]
y_raw = [row[-1] for row in training_data]
encoder = OrdinalEncoder()
X = encoder.fit_transform(X_raw)
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y_raw)
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
```

```
model.fit(X, y)
new_sample = ['Overcast', 'Cool', 'Normal', 'Weak']
new_encoded = encoder.transform([new_sample])
prediction = label_encoder.inverse_transform(model.predict(new_encoded))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Decision Tree Classification Using the ID3 Algorithm was implemented for play tennis prediction and the new sample was classified using the trained model.

Problem 2: Student Pass Prediction

Aim

To implement Decision Tree Classification Using the ID3 Algorithm for student pass prediction and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Attendance, StudyHours, Assignment and their class labels to train a model, then classify a new sample.

Algorithm

6. Import the required Python libraries.
7. Store the training samples in a feature matrix X and labels in y.
8. Create the classifier and fit it using X and y.
9. Prepare the new sample and apply the same preprocessing, if required.
10. Predict and display the class of the new sample.

Input (Training Data)

Attendance	StudyHours	Assignment	Target
High	High	Complete	Pass
High	Low	Complete	Pass
Low	Low	Incomplete	Fail
Medium	High	Complete	Pass
Low	Medium	Incomplete	Fail
Medium	Low	Complete	Pass
Low	High	Incomplete	Fail
High	Medium	Complete	Pass

Program

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
import numpy as np

attributes = ['Attendance', 'StudyHours', 'Assignment']
training_data = [['High', 'High', 'Complete', 'Pass'], ['High', 'Low', 'Complete', 'Pass'], ['Low', 'Low',
'Incomplete', 'Fail'], ['Medium', 'High', 'Complete', 'Pass'], ['Low', 'Medium', 'Incomplete', 'Fail'],
['Medium', 'Low', 'Complete', 'Pass'], ['Low', 'High', 'Incomplete', 'Fail'], ['High', 'Medium',
'Complete', 'Pass']]
X_raw = [row[:-1] for row in training_data]
y_raw = [row[-1] for row in training_data]
encoder = OrdinalEncoder()
X = encoder.fit_transform(X_raw)
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y_raw)
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
```

```
model.fit(X, y)
new_sample = ['Low', 'Medium', 'Incomplete']
new_encoded = encoder.transform([new_sample])
prediction = label_encoder.inverse_transform(model.predict(new_encoded))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Decision Tree Classification Using the ID3 Algorithm was implemented for student pass prediction and the new sample was classified using the trained model.

Problem 3: Loan Approval Classification

Aim

To implement Decision Tree Classification Using the ID3 Algorithm for loan approval classification and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Income, CreditHistory, Employment and their class labels to train a model, then classify a new sample.

Algorithm

11. Import the required Python libraries.
12. Store the training samples in a feature matrix X and labels in y.
13. Create the classifier and fit it using X and y.
14. Prepare the new sample and apply the same preprocessing, if required.
15. Predict and display the class of the new sample.

Input (Training Data)

Income	CreditHistory	Employment	Target
High	Good	Stable	Yes
High	Poor	Stable	No
Low	Good	Stable	Yes
Low	Poor	Unstable	No
Medium	Good	Stable	Yes
Medium	Poor	Unstable	No
High	Good	Unstable	Yes
Low	Good	Unstable	No

Program

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
import numpy as np

attributes = ['Income', 'CreditHistory', 'Employment']
training_data = [['High', 'Good', 'Stable', 'Yes'], ['High', 'Poor', 'Stable', 'No'], ['Low', 'Good', 'Stable',
'Yes'], ['Low', 'Poor', 'Unstable', 'No'], ['Medium', 'Good', 'Stable', 'Yes'], ['Medium', 'Poor', 'Unstable',
'No'], ['High', 'Good', 'Unstable', 'Yes'], ['Low', 'Good', 'Unstable', 'No']]
X_raw = [row[:-1] for row in training_data]
y_raw = [row[-1] for row in training_data]
encoder = OrdinalEncoder()
X = encoder.fit_transform(X_raw)
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y_raw)
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
model.fit(X, y)
```

```
new_sample = ['Medium', 'Good', 'Stable']
new_encoded = encoder.transform([new_sample])
prediction = label_encoder.inverse_transform(model.predict(new_encoded))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Decision Tree Classification Using the ID3 Algorithm was implemented for loan approval classification and the new sample was classified using the trained model.

Problem 4: Crop Type Recommendation

Aim

To implement Decision Tree Classification Using the ID3 Algorithm for crop type recommendation and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Rainfall, Soil, Season and their class labels to train a model, then classify a new sample.

Algorithm

16. Import the required Python libraries.
17. Store the training samples in a feature matrix X and labels in y.
18. Create the classifier and fit it using X and y.
19. Prepare the new sample and apply the same preprocessing, if required.
20. Predict and display the class of the new sample.

Input (Training Data)

Rainfall	Soil	Season	Target
High	Loamy	Monsoon	Rice
High	Clay	Monsoon	Rice
Low	Sandy	Summer	Millet
Medium	Loamy	Winter	Wheat
Low	Loamy	Winter	Wheat
Medium	Clay	Monsoon	Rice
Low	Sandy	Winter	Millet
Medium	Loamy	Summer	Maize

Program

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
import numpy as np

attributes = ['Rainfall', 'Soil', 'Season']
training_data = [['High', 'Loamy', 'Monsoon', 'Rice'], ['High', 'Clay', 'Monsoon', 'Rice'], ['Low',
'Sandy', 'Summer', 'Millet'], ['Medium', 'Loamy', 'Winter', 'Wheat'], ['Low', 'Loamy', 'Winter',
'Wheat'], ['Medium', 'Clay', 'Monsoon', 'Rice'], ['Low', 'Sandy', 'Winter', 'Millet'], ['Medium', 'Loamy',
'Summer', 'Maize']]
X_raw = [row[:-1] for row in training_data]
y_raw = [row[-1] for row in training_data]
encoder = OrdinalEncoder()
X = encoder.fit_transform(X_raw)
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y_raw)
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
```

```
model.fit(X, y)
new_sample = ['High', 'Clay', 'Monsoon']
new_encoded = encoder.transform([new_sample])
prediction = label_encoder.inverse_transform(model.predict(new_encoded))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Decision Tree Classification Using the ID3 Algorithm was implemented for crop type recommendation and the new sample was classified using the trained model.

Problem 5: Machine Fault Detection

Aim

To implement Decision Tree Classification Using the ID3 Algorithm for machine fault detection and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Temperature, Vibration, Noise and their class labels to train a model, then classify a new sample.

Algorithm

21. Import the required Python libraries.
22. Store the training samples in a feature matrix X and labels in y.
23. Create the classifier and fit it using X and y.
24. Prepare the new sample and apply the same preprocessing, if required.
25. Predict and display the class of the new sample.

Input (Training Data)

Temperature	Vibration	Noise	Target
High	High	Loud	Fault
High	Low	Loud	Fault
Normal	Low	Quiet	Normal
Normal	High	Quiet	Normal
High	High	Quiet	Fault
Low	Low	Quiet	Normal
High	Medium	Loud	Fault
Normal	Medium	Loud	Normal

Program

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
import numpy as np

attributes = ['Temperature', 'Vibration', 'Noise']
training_data = [['High', 'High', 'Loud', 'Fault'], ['High', 'Low', 'Loud', 'Fault'], ['Normal', 'Low',
'Quiet', 'Normal'], ['Normal', 'High', 'Quiet', 'Normal'], ['High', 'High', 'Quiet', 'Fault'], ['Low', 'Low',
'Quiet', 'Normal'], ['High', 'Medium', 'Loud', 'Fault'], ['Normal', 'Medium', 'Loud', 'Normal']]
X_raw = [row[:-1] for row in training_data]
y_raw = [row[-1] for row in training_data]
encoder = OrdinalEncoder()
X = encoder.fit_transform(X_raw)
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y_raw)
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
model.fit(X, y)
```

```
new_sample = ['High', 'Low', 'Loud']
new_encoded = encoder.transform([new_sample])
prediction = label_encoder.inverse_transform(model.predict(new_encoded))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Decision Tree Classification Using the ID3 Algorithm was implemented for machine fault detection and the new sample was classified using the trained model.

Conclusion

Five different application problems were demonstrated using Decision Tree Classification Using the ID3 Algorithm. The results show how a trained machine-learning model can use example data to classify a new observation. Performance depends on the quality and size of the training data.

Requirements

Python 3, scikit-learn, NumPy, and a Python IDE or Jupyter Notebook.